

Assignment 2 Machine Learning 1



NIA'S:

100495777 César Ramírez Chaves

100496380 Jesús Navarro Benito

1. Updating the Q_table:

In order to build the function that updates our Q_table, we recycled the one we used for Tutorial 4, the only challenge we faced was to decide which was going to be our terminal state, but we figured out we could import the function "check_game_over" from the "SnakeEnv" and with all this our Qtable was being updated properly

2. Updating the main function:

Here we only needed to implement minor changes, such as defining the number of states and episodes, here we have chosen 100.000 for the training.

Other implementations were choosing the appropriate action based on the Q-Learning method, the current state, and the next state, and finally computing the total reward.

3. Updating the reward function:

For the first phase, we had to build a state and reward function, to begin with, we only wanted to test our agent without growing to see if we managed to complete the Q-Learning task. Our first obstacle was to assign rewards, as we wanted the agent to go for the food without colliding with the borders or itself, so as we had no growing we could assign the same negative reward for dying by having collided with a border or itself. Then we assigned the same positive reward to eat the food. Finally, we assigned a positive reward of ten percent of the others as we wanted our agent to be alive for the most time possible.

For the second phase, we had to build a more complex function, so what we did in the very first iterations was to use a similar structure as before, due to the fact that the agent is choosing the actions completely random it is more likely to die without eating more than two or three apples. Then, as we decreased the learning rate, we increased the negative rewards in order to avoid collisions with itself and locking up.

4. Updating the get state function:

Moving forward, with the state function, we encountered a more feared challenge, this was caused by a misunderstanding issue with the representation of the state itself, it was not until we discovered that each state represented a scenario with all four possible actions our agent could follow, that we could not overcome this first issue. Once this was solved, we figured out that as had happened before in Assignment 1 the relative position of head and food turned out to work well, which led us to 8 different states, but this was not enough for our agent to learn properly, that is why we also made a distinction between the X_distance and the Y_distance, now we had 24 states, but somehow it did not work as expected, and it all was clear when we took a look into the Q_table that we figured out that some of those states where

unreachable, so when got rid of those we were left with only 16. So far we have managed to make some improvements, as our agent was able to learn and was going for the food, but it kept making strange movements, that is when we decided to introduce directions in our states, which led to 64 states, and also we decided to aggregate a new change in our reward function, we were observing that our agent was making those strange movements as it had a positive reward for being alive, and once we changed it to a negative reward our agent went straight forward for the food now

For the second phase, the 64 states were not enough, to avoid their own collisions, so what we thought, was to include for each state we previously had, the eight different possibilities of movement that the agent has in each state, which resulted in 512 states, with this new state space we have overcome the collision with itself issue, but not the locking up problem, for this we have some ideas that we have not been able to implement in our project due to time and complexity.

5. Our thinking process

In our thinking process, we divided the assignment into 3 phases. The first one was to make the snake work, meaning that the snake basically moves and is represented on the screen. The second one was to make the snake go to the food, which we did successfully. And the last one was to make the snake avoid itself which we more or least did. We will explain all this in-depth now.

What we did first to make the snake go to the food in terms of states is the following ...

After achieving that the snake goes towards the food with those states we moved on with trying to make the snake avoid itself (note that at first we did all these to make the snake go to the food without growing body) for that at first we played with the reward function, we first penalized the snake more to for dying than eating the food, for instance, -300 for dying, 200 for eating an apple and -3 for every time it moves and doesn't eat an apple, with the objective of making the snake take the optimal path; and the snake more or least knew that it just needed to go to the apple. But without a growing body, it was just impossible for the snake to collide with itself so from now on we needed to implement a growing body.

So we tried with the growing body, the same rewards and the snake actually did go to the food but it didn't avoid itself, so we decided to penalize the snake a lot more for dying and we found out that these actually don't work because the snake now only makes movements in circles without chasing the food, it is like afraid to die or something like that, so we reduced the penalization again to -300 and raised the reward for eating an apple to 300, but we got the same result, it goes to the apple but collides with itself again.

After playing for a while with these parameters and adding some functions like, collision with wall and collision with itself, and putting more penalization to colliding with itself, or when the snake moves away from the apple and doesn't have collision is also penalized with negative

reward, we noticed that nothing changed and we started to wonder that maybe the reward is not the problem and the problem is that the states that we had (64 states) just didn't have the sufficient scenarios so the snake couldn't learn and didn't actually know what to do if it finds a piece of its body in front. So we came up to the conclusion at this point that the snake just didn't know what to do in those situations and it was moving due to relative position and not due to avoid collision as we didn't have scenarios in the states for that.

So after coming up with those conclusions we decided to make more states, so we analyzed which scenarios can there be and which actions can the snake take in those situations. For these we used `check_collision`, if there was no collision then we have the states that we already have, but if there is a collision (imagine we are going to the right and we check collision) if there is collision to the right then we will have 4 more possibilities (collision up and collision down), (collision up but no collision down), (not collision up but collision down) and (no collision neither up nor down), notice that if we are going to the right we cannot go to the left, therefore, we didn't take into account those options because they are impossible, therefore we would have 64×5 states which are 320 states. Now we trained this model at first with 10k episodes of training and we noticed that there were some 0's in the q-table and we didn't know why because on the paper the snake was able to make those moves, but what happened is that we had so many states and with 10k episodes was not enough so we made 100k episodes for exploring and now the q-table was good, we made at first a simple reward just penalizing the snake when colliding and moving and rewarding when eating. In this attempt the snake was chasing the food as always but was not avoiding itself as we expected, the snake took longer paths to be safer I guess but was not avoiding itself when the moment of the possible collision arrived, therefore we came up to the conclusion that rather the snake was suicidal or we were doing something wrong.

After checking the code and making some drawings we noticed that there were some cases that we were not taking into account, those cases were, If the snake was going to the right we were checking collision to the right but it is not a smart move at all, because yes the snake is moving to the right but imagine now needs to turn up to chase the apple, if there is body when goes up it is going to die because we are not checking collision up we are checking the collision to the right, therefore what we did is regarding the position of the apple we check the collision in one place or another. Imagine we need to go to the left no matter in which direction are we going, therefore what we need to do is check the collision in that direction, if there is no collision then we assign a state, but if there is a collision we need to check the collision in the other 2 possible directions, imagine we are going to the right and we need to go Up, then we check collision up and if there is collision up what we are going to do is check collision to the right and down as these are the only 2 possible directions, because if we are going to the right then we cannot change directions immediately to the right.

After running this model that I already explained we checked it with different rewards and noticed it does not change a lot so we remained with the simple rewards described previously, now finally the snake avoids itself in the vast majority of the scenarios, up to a point that many times dies because the snake ends up without possible movements as we have not taught it to go to scenarios where it could die. But that is something we did not have time for.

We also had another environment with 512 states, for which we implemented 8 different scenarios for the 64 states we previously had for the not growing. Then for the learning of the agent, we decreased the alpha value each time we ran the simulation, we also increased the negative rewards in the final simulations as we wanted the agent to avoid the self collisions. Although we have achieved the agent to prevent itself many times sometimes the game itself collides and dies without any reason. What we have not managed to avoid locking up.

Furthermore, for the reward function, in this particular case we have given a positive reward for eating food and having 1 or more available directions, and negative feedback if after eating there is no space to move or if it dies, if it dies, we have given an increasing negative reward in order of the available directions. What we expected from this is that the agent avoided locking up itself.

6. Conclusions

After trial and error many many times we achieved a decent model where the snake finally achieved, first going towards the food, and second avoiding itself. What we have seen is that for the “Not growing” agent with only 64 states, we had all possibilities covered, but this was not the case for the “Growing” agent, in which we needed 512 states and a more complex reward function. What we expected from this project was for the snake to overcome the main issue we had with the “move_tutorial” we designed in the past projects, to avoid locking up itself, but we haven’t. We believe that the main reason why we have not been able to succeed in this goal is due to the higher complexity of the game, as we are incapable of plotting the size of the snake into the state space, if we managed to have more states based on the size of the snake and its relative position with itself, we could solve this problem. From this project, we can say that we are satisfied with what we have, even though we have not solved that problem, we have managed to overcome other issues, such as avoiding colliding with itself or knowing how to disentangle from itself

